Technical Design Document – `wego`

> **Author**: *Automated Static■Analysis Tool*

> **Date**: 2026■11■12

> **Repository**: `github.com/schachmat/wego`

> **Disclaimer**

> This document is generated solely from the static artifacts supplied in the prompt.

> Any information not present in those artifacts is marked **“Not detected from static analysis.”**

> No assumptions, guesses, or fabricated details are included.

---

1. Summary

`wego` is a minimal Go■based project that appears to provide a terminal■based user interface.

The repository contains three primary packages:

Package	Purpose (as inferred)	Evidence
`backends`	Backend logic	Directory listed in the repository tree
`frontends`	Frontend/UI logic	Directory listed in the repository tree
`iface`	Shared interfaces between backends and frontends	Directory listed in the repository tree

The project is built with **Go 1.27** and pulls in a handful of third■party libraries that support terminal rendering and color handling.

---

2. Scope

Item	Status
Application domain	Terminal UI application (inferred)
Persistence	None (no database tables, no persistence configuration)
Authentication / Authorization	None (no evidence of auth mechanisms)
Deployment	Not defined (no Dockerfile, no Kubernetes manifests, no CI/CD workflow content)
External services	None (no external API calls, no imports listed)

---

3. Technology Stack

Layer	Technology	Evidence
Language	Go 1.27	`go.mod` specifies `go 1.27`

| **Terminal UI** | `github.com/mattn/go-colorable`, `github.com/mattn/go-runewidth`, `github.com/muesli/mango`, `github.com/muesli/roff` | Listed in `go.mod` |

| **Utility** | `github.com/schachmat/ingo` | Listed in `go.mod` |

| **Indirect Dependencies** | `github.com/clipperhouse/uax29/v2`, `github.com/mattn/go-isatty`, `golang.org/x/sys` | Listed as indirect in `go.mod` |

| **Build Tool** | Go modules | `go.mod` present |

| **CI** | Not detected | `.github/workflows/` exists but contents not provided |

| **Configuration** | Not detected | No `.yml` or `.json` files with configuration content |

| **Testing** | Not detected | No test files or evidence of tests |

---

4. Repository Structure

```
wego/
■■■ .github/
■   ■■■ workflows/          # CI/CD workflows (content not provided)
■■■ backends/               # Backend logic (source files not provided)
■■■ frontends/              # Frontend/UI logic (source files not provided)
■■■ iface/                  # Shared interfaces (source files not provided)
■■■ go.mod                  # Go module definition
■■■ go.sum                  # Go module checksum (not shown)
■■■ *.go                    # 15 Go source files (names not listed)
■■■ *.md                    # 3 Markdown files (names not listed)
■■■ *.yml                   # 3 YAML files (names not listed)
```

*All file names and contents beyond the ones explicitly listed are **Not detected from static analysis**.*

---

5. Architecture

The architecture is a simple **monolithic** Go application with a clear separation of concerns:

1. **Frontends** – UI components that render to the terminal.

2. **Backends** – Business logic and data handling.

3. **Iface** – Interface definitions that decouple frontends from backends.

> **Flow**

> Frontend modules call backend functions through the interfaces defined in `iface`. The backends perform the core logic and return results to the frontends for display.

---

6. Components

| Component | Responsibility | Evidence |

|-----------|----------------|----------|

| `backends` | Implements application logic | Directory present |

| `frontends` | Renders UI, handles user input | Directory present |

| `iface` | Defines contracts between frontends and backends | Directory present |

| `go.mod` | Dependency and module management | File content provided |

| `go.sum` | Dependency checksums | Not provided (assumed present) |

> **Unknowns** – The internal structure, functions, and data types within each package are **Not detected from static analysis**.

---

7. Flows

1. **Startup** – The Go binary is built from the source files.
2. **UI Rendering** – Frontend modules use `mango`/`roff` for terminal rendering.
3. **Business Logic** – Backend modules process user actions via interfaces.
4. **Termination** – Application exits gracefully (no explicit shutdown logic detected).

---

8. APIs

> **Not detected from static analysis**

> No HTTP endpoints, gRPC services, or other external APIs are defined in the provided artifacts.

---

9. Data

Data Layer	Status
In■memory	Likely, but no data structures are visible
Persistent	None (no database tables, no persistence config)
External	None (no external data sources)

> **Unknowns** – The shape of any in■memory data structures is **Not detected from static analysis**.

---

10. Dependencies

Dependency	Purpose	Version	Evidence
`github.com/mattn/go-colorable`	Terminal color support	v0.1.15	`go.mod`
`github.com/mattn/go-runewidth`	Rune width calculations	v0.0.30	`go.mod`
`github.com/muesli/mango`	Terminal UI rendering	v0.2.0	`go.mod`
`github.com/muesli/roff`	Rich text formatting	v0.1.0	`go.mod`
`github.com/schachmat/ingo`	Utility library (unknown)	v0.0.0-20170403011506-a4bdc0729a3f	`go.mod`
Indirect	`github.com/clipperhouse/uax29/v2`, `github.com/mattn/go-isatty`, `golang.org/x/sys`	Indirect	`go.mod`

---

11. Configuration

Config Type	Status
Build	`go.mod` defines module and Go version
Runtime	None detected
CI/CD	